Comparison of Bayes Point Machine with Support Vector Machine for binary classification

Gang Wu
Department of Electrical and Computer Engineering, UCSB
gwu@engineering.ucsb.edu

Edward Chang
Department of Electrical and Computer Engineering, UCSB
echang@ece.ucsb.edu

Huaxin You
Department of Statistics and Applied Probability, UCSB
hxyou@pstat.ucsb.edu

Abstract

The Bayes point machine(BPM) is a new learning algorithm for kernel classifiers rooted in the Bayesian learning theory. Compared with current popular kernel classifiers such as Support Vector Machine(SVM), Bayes pint machine has been theoretically demonstrated to have better learning ability. We compare these two classification techniques by providing several experiments on some sets of artificial data and standard UCI benchmarking datasets. At the same time, we study the relationship between misclassification rate of testing data and bounce steps in billiard ball algorithm, which is used to estimate the Bayes point in version space. Traditionally, the BPM algorithm chooses a pre-specified threshold(TOL), usually being 0.01 or 0.001, as the stopping criterion. Experimentally, we find that misclassification rate for a set of testing data will be overfitting after certain bounce steps, although at the time, the TOL of the billiard algorithm is much smaller than pre-specified one. We shown this method saves much training time.

1. Introduction

In this paper we compare the learning ability of the SVM and BPM. SVM(Support vector machine). SVM is a type of learning machine based on statistical learning theory. It can be thought as an alternative training technique for Polynomial, Radial Basis Function and Multi-Layer Perceptron classifiers, in which solution for the weights of network are treated as a quadratic programming with a set of constraints. From a Bayesian perspective, SVM chooses the hypothesis corresponding to the largest possible hyperplane that can be inscribed in version space [1].

However, BPM is a novel type of kernel classifier, based on the Bayesian learning theory. It approximates the Bayes-optimal decision by the center of mass of version space [1]. This alternative approach constructs the hypothesis using the whole of version space, not just using a largest inscribable hyerplane ball in SVM. One can image that if this inscribable ball can not cover most volumes of version space, SVM would get an unreasonable kernel functions We can see it from Fig.1.

However, BPM often suffers from the demanding computational cost of sampling the Bayesian posterior. In this paper, we suggest an alternative stopping criterion for BPM billiard algorithm by studying the relationship between misclassification rate of testing data and training steps of billiard algorithm. It's experimentally shown that this technique can save a great deal of training time.

The plan of the paper is as follows: in the next section we briefly sketch the ideas underlying SVM and BPM, and see their differences by presenting several experiments. In section 3, we study the relationship between misclassification rate and training steps for BPM billiard algorithm, and present a reasonable but effective stopping criterion for it. Last section summarizes the paper.

2. Algorithm

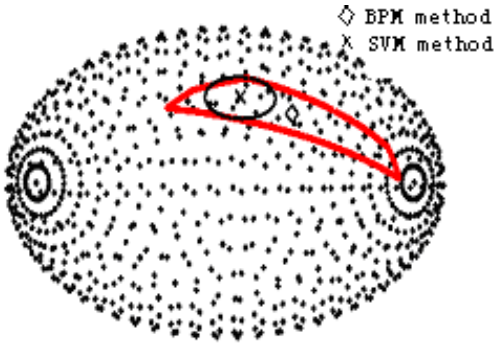


Figure 1: Version space V(z) for a 3D toy dataset. From the Bayesian perspective, one can see that SVM approximates the Bayes point(diamond, trained by BPM) by the center of the largest inscribable ball in the version space. Apparently, BPM is more reasonable.

2.1 Support Vector Machine

In this section we briefly show how SVM classification leads to the formulation of a QP programming problem for solving a set number of the weights that is equal to the number of data points. A more detailed description of SVM can be found in [2]. Let $\{(x_i, y_i)\}_{1\le i\le \ell}$ be a set of training samples, where $x_i \in \Re^n$ and $y_i \in \{-1,1\}$. Then we map these samples into a feature space using a kernel function $K(x_i, x_j)$ that defines an inner product in this space. Now we wish to find one linear classifier that minimizes the generalization error, or at least an upper bound on it. Vapnik showed [2] that in this situation, the corresponding hyperplane leaves the maximum margin between the two classes, where the margin is defined as the distance between the canonical hyperplane (w, b) and the closest of the data points, which is $\frac{2}{\|w\|}$.

At this point, this problem can be solved using standard QP optimization techniques as the follows:

$$W(a) = \sum_{i=1}^{\ell} a_i - \frac{1}{2}\sum_{i,j=1}^{\ell} a_i a_j y_i y_j K(x_i, x_j) \quad (1)$$

under the constraints

$$\sum_{i=1}^{\ell} a_i y_i = 0 \text{ and } a_i \ge 0, i = 1,..., \ell. \quad (2)$$

Then the decision function given by an SVM is:

$$f(x) = sign\left(\sum_{i=1}^{\ell} a_i^* y_i K(x_i, x) + b\right), \quad (3)$$

where a_i^* are got by maximizing (2). Those points that satisfy $y_i f(x_i) = 1$ are called support vectors.

2.2 Bayes Point Machine

We introduce some standard notation for BPMs; for a detailed description, see [3].

2.2.1 The Bayes Classification Strategy

For a new testing point *x* and each class *y*, the *Bayes classification strategy* chooses the minimum expected loss incurred by each hypothesis $h \in H$ applied to *x* and weights it according to its posterior probability $P_{H|Z^m=z}(h)$. The final decision is given by

$$Bayes_z(x) = \text{argmin}_{y\in Y} E_{H|Z^m=z}[l(H(x), y)] \quad (4)$$

2.2.2 The Bayes Point

(4) suffers from the demanding computational cost as it has to evaluate $P_{H|Z^m=z}$ at each possible vector (x, y). [3] suggests to require the classifier to lie within a fixed hypothesis space. Then the evaluation of $P_{H|Z^m=z}$ at one test point *x* could be faster.

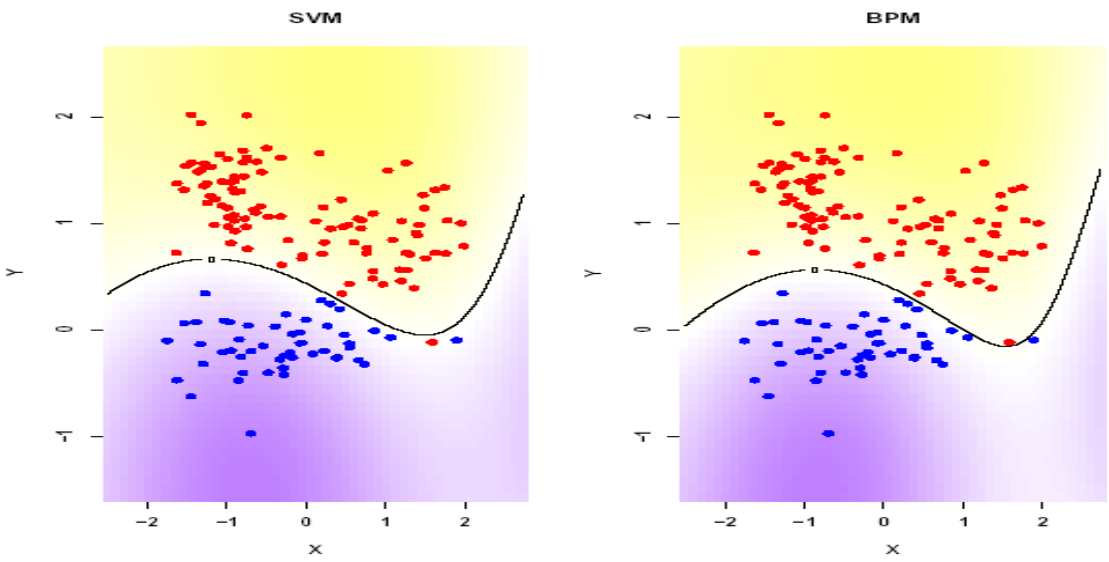


Figure 2: Binary classification for a 2D toy dataset. We choose linear kernels function for a support vector machine (**left**) and Bayes point machine (**right**) using hard margins (λ =0). One can see that support vector machine chooses classifiers with the largest margin.

At the time, the Bayes point algorithm h_{bp} is given by

$$h_{bp}(z)=\arg\min_{h\in H} E_X\left[E_{H|Z^m=z}\left[l(h(X),H(X))\right]\right], \quad (5)$$

The classifier $h_{bp}(z)$ is called the *Bayes point*, which approximates best the (4).

2.2.3 The Center of Mass

Calculation of (5) requires to know the input distribution P_X .Therefore, we use the center of mass

$$w_{cm}=\frac{E_{W|Z^m=z}[W]}{\left\|E_{W|Z^m=z}[W]\right\|} \quad (6)$$

as a good approximation to the Bayes point (5). Then the main goal of BPM algorithm is to calculate the center of mass w_{cm}. Ralf[3] presented a billiard-like algorithm based on a first order Markov chain. For detailed billiard algorithm, see [3].

2.2.4 Stopping Criterion

In [3], Ralf suggests computing an upper bound on ρ_2, the weighting factor of the new part of the trajectory. If this value falls below a pre-specified threshold (TOL), [3] stops the algorithm. Usually, TOL is set as 0.01 or 0.001.

3. Experiment

3.1 Version Space

For demonstrating the potential benefits of a Bayes point machine over a support vector machine, we setup a 3D toy dataset of 50 training points in $\Re^3$. These data are randomly allocated to one of two classes, -1 and 1. We show its version space *V*(*z*) in Fig. 1, which is an elongated one surrounded by red bold boundaries. The inscribable ball in *V*(*z*) is just the hypersphere corresponded to SVM.

However, BPM utilizes the whole version space to estimate the Bayes point, which is the diamond in Figure 1. Therefore, BPM results in the great reduction in generalization error, in this example, which is 4.6% for BPM solution and 10.3% for SVM method. Here, we choose linear kernel function and hard margin for both solutions

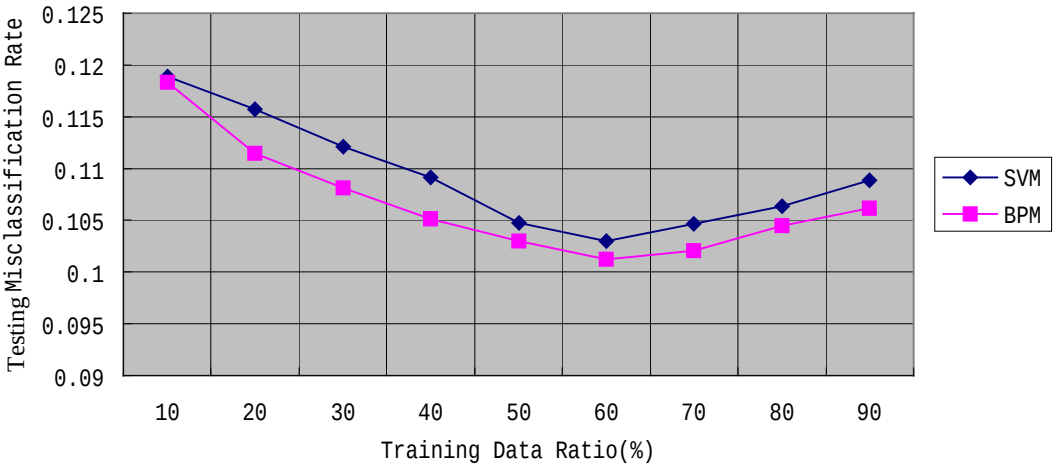


Figure 3 Misclassification rates of testing data as a function of the number of training data for UCI **waveform** data set using SVM and BPM solutions. Here, we choose RBF kernel functions with $\sigma = 1.0$ and run data for 7 times. Notice, testing misclassification rate reaches the lowest when training data ratio is 60%.

3.2 Binary Classification

At first, we set up a 2D toy dataset of 146 training data in $\Re^2$. The data points are randomly generated to one of two classes, -1 and 1. We choose a RBF as kernel function for both SVM and BPM with $\sigma = 2.0$. One can see their differences for binary classification from Fig.2. In SVM solution, one training data is apparently mis-classified. However, the BPM solution appears more reasonable and smoother than SVM one.

3.3 UCI Benchmark Datasets

Now, we compare the learning performance of two solutions on real world datasets. At first, we randomly select a set of data as training data from **waveform** dataset, from 10% to 90%, then use other ones as testing data to calculate their classification error. Here, RBF kernel with $\sigma = 1.0$ is chosen. Total size of data is 3304. Since training time of BPM algorithm is very long, we have to run just 10 times for each set of training data. From Figure 3, One can see that the Bayes point machine always gets better results than SVM. At the same time, one should note that misclassification rate of testing data is smallest when training data ration is 60%. This is because the training data are sampled without replacement. More details could be referred to [4].

Table 1 shows comparison results of 5 UCI benchmark datasets, **tic-tac-toe**, **SPECT**, **Liver-disorders**, **Ionosphere** and **Dermatology**. For each case, one set of data(which is 10%, 30%, 50%, 60% or 80%) are partitioned as training data and others are for testing. By using SVM and BPM solutions, we compare their misclassification rate. This time, RBF kernel with $\sigma = 3.0$ is chosen. From this table, we can get the same conclusions as the previous. (1). BPM outperforms SVM. (2). 60% is an optimal sampling rate for replacement sampling. Note that smaller rates require less computation.

3.4 Bounce Steps for Billiard Algorithm

As in the subsection 2.2.4., Billiard algorithm always chooses a pre-specified threshold(TOL) as stopping criterion., which is often 0.01 or 0.001. However, it always

	SVM					BPM				
	10%	30%	50%	60%	80%	10%	30%	50%	60%	80%
Tic-tac-toe	0.112	0.0921	0.0858	0.0781	0.0890	0.0732	0.0675	0.0620	0.0580	0.0652
spect	0.301	0.2934	0.2902	0.2888	0.2934	0.2902	0.2834	0.278	0.272	0.2811
Liver-disorder	0.5126	0.470	0.4436	0.4333	0.4755	0.4742	0.4412	0.4311	0.4167	0.4434
Derm-atology	0.0272	0.0256	0.0243	0.0236	0.0250	0.0260	0.0233	0.0210	0.0204	0.0218
Ionosphere	0.1920	0.1884	0.1853	0.1844	0.1876	0.1647	0.1536	0.1501	0.1489	0.1523

Table 1: Experimental results on five benchmark datasets. Shown is the testing misclassification rate under different percents of training data. Each result is obtained as the average of 7 different runs. One can see that BPM always get lower misclassification rate than SVM. Here, RBF kernel functions with $\sigma = 1.0$ are chosen.

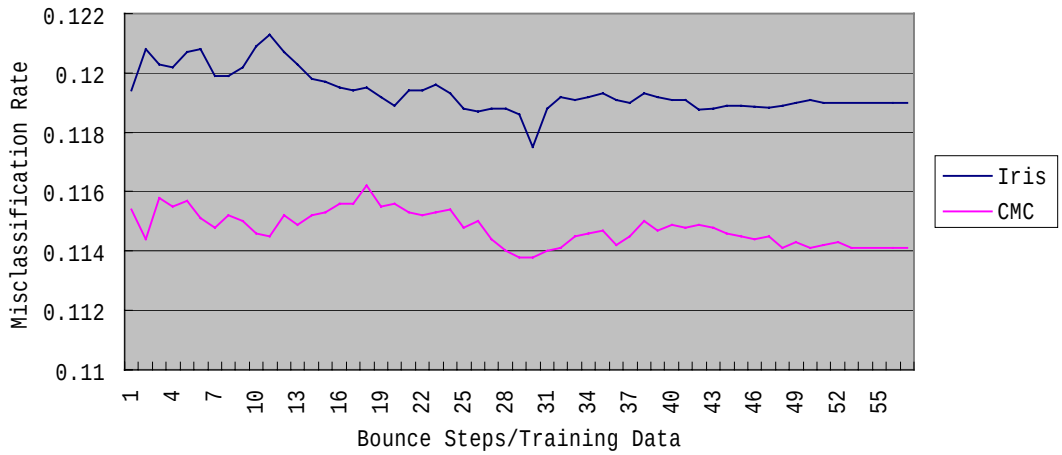


Figure 4 Misclassification rate as a function of bounce steps in BPM billiard algorithm.

takes an intolerably long time to satisfy this criterion. In our experiment, we find that misclassification rate will be basically overfitting when bounce steps of billiard ball achieves some certain values. To illustrate this, we choose **Iris** and **CMC** from UCI benchmark datasets. For each case, data set is randomly split into a training data and a testing data in the ratio 60%:40% and trained for 10 times. The results are shown in Fig.4. One can see that when bounce steps (m)achieves 50 times the number of training data(n), misclassification rate of testing data is overfit. Actually, when $m = 30n$, the current misclassification rate is tolerable.

Meanwhile, we find the current ρ_2 is much smaller than TOL, and running time is only a small part of the whole time when ρ_2 falls into TOL. To better illustrate this point, we add other 3 datasets, **liver-disorder**, **tic-tac-toe** and **waveform** to study the relationship between bounce steps and misclassification rate. Table 2 shows each ρ_2, running time and misclassification rate at different bounce

	$m = 30n$	$m = 40n$	$m = 50n$	$m = 60n$	$m = 110n$	running time	TOL
Waveform	0.0325	0.0246	0.0197	0.0165	0.0108	1400m	0.01
	0.1027	0.1020	0.1035	0.1027	0.1027		
	320m	470m	550m	680m	1200m		
Iris	0.003	0.0022	0.0018	0.0015	0.0010	30s	0.001
	0.1175	0.1191	0.119	0.119	0.119		
	9s	12s	16s	19s	30s		
Tic-tac-toe	0.0043	0.0033	0.0026	0.0022	0.0012	100m	0.001
	0.1016	0.1016	0.1016	0.1016	0.1016		
	21m	28m	34m	43m	80m		
CMC	0.1138	0.1149	0.1141	0.1141	0.1141	11m	0.01
	0.0362	0.0263	0.0216	0.0195	0.01		
	3m	4m	5m	6m	11m		
Liver	0.0048	0.0036	0.0029	0.0024	0.0013	30s	0.001
	0.4167	0.4333	0.4333	0.4333	0.4333		
	6s	8s	11s	13s	23s		

Table 2: Comparison of misclassification rate, ρ_2 value and running time at different bounces steps on five benchmark datasets. Here, RBF with $\sigma = 1.0$ is chosen as kernel function. Results show that when billiard algorithm is trained after *m* (bounce steps) = 30~50*n* (number of training data) steps, misclassification rate of testing data is overfit.

steps. We can see that basically, replacing the original stopping criterion TOL=0.01 or 0.001 with *m*=30~50*n* doesn't affect performance of learning machine. Meanwhile, it save a great deal of running time. E.g., for waveform dataset, the whole running time for achieving TOL=0.01 is 1400m, when misclassification error is 0.1027. However, if algorithm is stopped after m=30n bounces steps, although ρ_2 is only 0.0325, misclassification has achieved the lowest (0.1027) and running time is just only 320m, about 22.9% of 1400m.

4. Conclusion and Future Work

In this paper, we compared the Bayes point machine with current popular kernel learning machine-the Support vector machine. We showed that BPM outperforms SVM by experimenting them on both artificial data and real world benchmark datasets.

For the long training time of BPM algorithm, we found that misclassification rate of testing data would be overfit after a certain of bounce steps. From experiments, we got an optimal training step for billiard ball algorithm. We also found that using this step as stopping criterion would dramatically reduce the training time for billiard ball algorithm.

Although this, compared with the Support vector machine, the Bayes point machine algorithm still takes a intolerably long training time. It's an interesting problem we should tackle in the future. Meanwhile, until now, we just utilize these two solutions for binary classification. In the future, we will compare their performance for multi-class image classification by incorporating some ensemble schemes, like OPC or PWC.

5. Reference

[1]. Ralf Herbrich, Thore Graepel, and Colin

Campbell. Bayes Point Machines. *Journal of Machine Learning Research*, 1:245-279,2001

[2]. V.Vapnik. *The Nature of Statistical Learning Theory*. Springer, 1995

[3]. V.Vapnik. *Statisctical Learning Theroy*. John Wiley & Sons, 1998

[4]. Ralf Herbrich, Thore Graepel, and Colin Campell. Bayes Point Machines: Estimating the Bayes Point in Kernel Space. *In Proceedings of IJCAI Workshop Support Vector Machines*, pages 23-27,1999

[5].Pàl Rujàn. Playing billiard in version space. *Neural Computation*, 9:99-122, 1997

[6]. UCI. University of California Irvine: Machine Learning Repository, 1990

[7]. O.Chapelle, V.Vapnik, O.Bousquet, and S.Mukherjee. *Choosing multiple parameters for support vector machines*. Machine Learning, 2001. in press

[8]. Edgar Osuna, Robert Freund, and Federico Girosi. An Improved Training Algorithm for Support Vector Machines. Proceedings of IEEE NNSP’97, Amelia Island, FL, 24-26 Sep.,1997